

ECS1005: Verilog HDL Mini Project Report

Student Name: Matthew Cooke
Student ID: 40450757

Project Number: 2
Project Title: Multi-Pattern 4-bit Serial Password Detector

By submitting the work, I declare that:

- o I have read and understood the University regulations relating to academic offences, including collusion and plagiarism. ([Academic Offences](#) | [Academic & Student Affairs](#) | [Queen's University Belfast](#))
- o The submission is my own original work and no part of it has been submitted for any other assignments, except as otherwise permitted.
- o All sources used, published or unpublished, have been acknowledged.
- o I give my consent for the work to be scanned using a plagiarism detection software

Table of Contents

LIST OF TABLES.....	3
LIST OF FIGURES	5
1. PROBLEM STATEMENT	9
2. THE FSM DESIGN	10
3. DUT VERIFICATION	13
REFERENCES	15

List of Tables

TABLE II.I

STATES OF THE FSM

State	Sequence Received
a idle	Reset
b found 0	0
c found 1	1
d found 01-10	01 or 10
e found 010-100	010 or 100
f found Null	2 bits recieved, not able to match either password
g found Null?	3 bits recieved, not able to match either password

TABLE II.II

ASSIGNING STATES TO A BINARY CODE

State	Binary Code
a	000
b	001
c	010
d	011
e	100
f	101
g	110

TABLE II.III
STATE TRANSITION TABLE

Current State	Input Sequence Bit x	Next State $D_A D_B D_C$	Output from Out z
$A \ B \ C$			
$a \ 0 \ 0 \ 0$	0	0 0 1	0
$0 \ 0 \ 0$	1	0 1 0	0
$b \ 0 \ 0 \ 1$	0	1 0 1	0
$0 \ 0 \ 1$	1	0 1 1	0
$c \ 0 \ 1 \ 0$	0	0 1 1	0
$0 \ 1 \ 0$	1	1 0 1	0
$d \ 0 \ 1 \ 1$	0	1 0 0	0
$0 \ 1 \ 1$	1	1 1 0	0
$e \ 1 \ 0 \ 0$	0	0 0 0	0
$1 \ 0 \ 0$	1	0 0 0	1
$f \ 1 \ 0 \ 1$	0	1 1 0	0
$1 \ 0 \ 1$	1	1 1 0	0
$g \ 1 \ 1 \ 0$	0	0 0 0	0
$1 \ 1 \ 0$	1	0 0 0	0
$h \ 1 \ 1 \ 1$	0	x x x	x
$1 \ 1 \ 1$	1	x x x	x

List of Figures

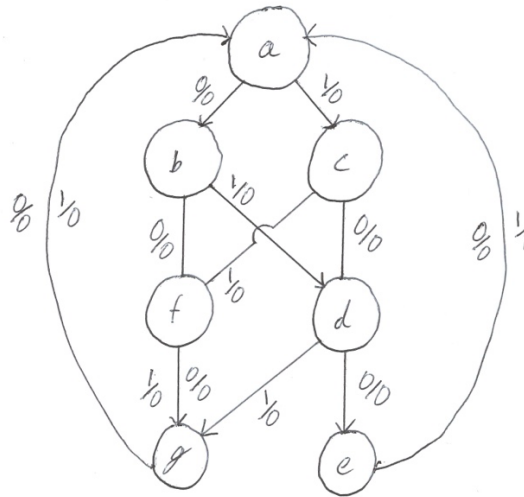


Figure 2.1. State diagram

Sequence Bit	0	1	0	1	0	0	1	0	1	0	0	1	0	1	0	0
state	b	d	e	a	b	f	g	a	c	d	e	a	b	d	g	a
fsmOut	0	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0

Figure 2.2. Dry run of the FSM

$$D_A = \sum m(2, 5, 6, 7, 10, 11) + \sum d(14, 15)$$

$\begin{matrix} C \\ X \\ A \backslash B \end{matrix}$	00	01	11	10
00	0	0	0	1
01	0	1	1	1
11	0	0	x	x
10	0	0	1	1

$$D_A = AC + CX' + A'Bx$$

$$D_B = \sum m(1, 3, 4, 7, 10, 11) + \sum d(14, 15)$$

$\begin{matrix} C \\ X \\ A \backslash B \end{matrix}$	00	01	11	10
00	0	1	1	0
01	1	0	1	0
11	0	0	x	x
10	0	0	1	1

$$D_B = AC + CX + A'B'x + A'BC'x'$$

$$D_C = \sum m(0, 2, 3, 4, 5) + \sum d(14, 15)$$

$\begin{matrix} C \\ X \\ A \backslash B \end{matrix}$	00	01	11	10
00	1	0	1	1
01	1	1	0	0
11	0	0	x	x
10	0	0	0	0

$$D_C = A'B'x' + A'B'C + A'BC'$$

$$f_{smOut} = \sum m(9) + \sum d(14, 15)$$

$\begin{matrix} C \\ X \\ A \backslash B \end{matrix}$	00	01	11	10
00	0	0	0	0
01	0	0	0	0
11	0	0	x	x
10	0	1	0	0

$$f_{smOut} = AB'C'x$$

Figure 2.3. Logic minimisation using Karnaugh maps

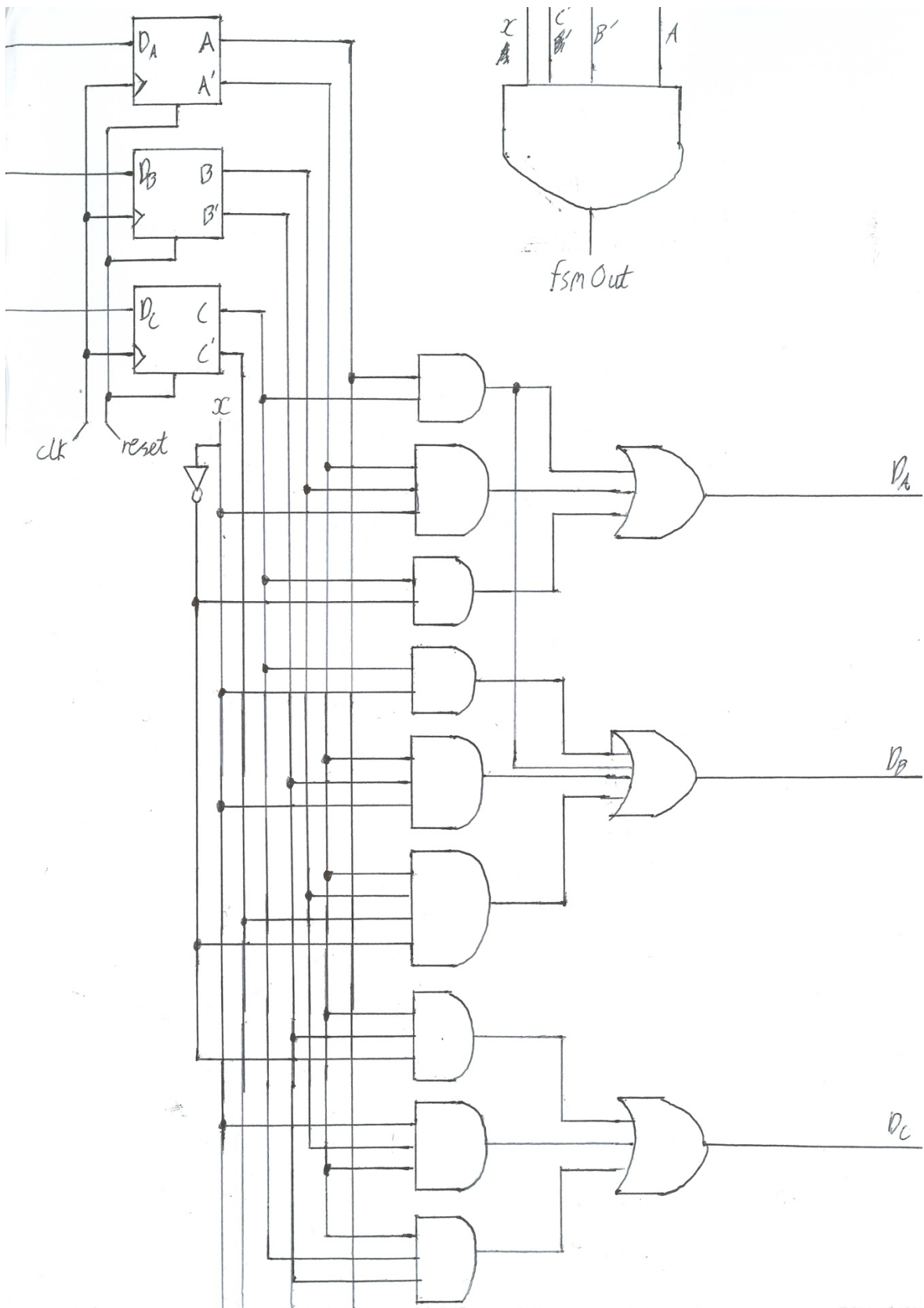


Figure 2.4. Final FSM circuit

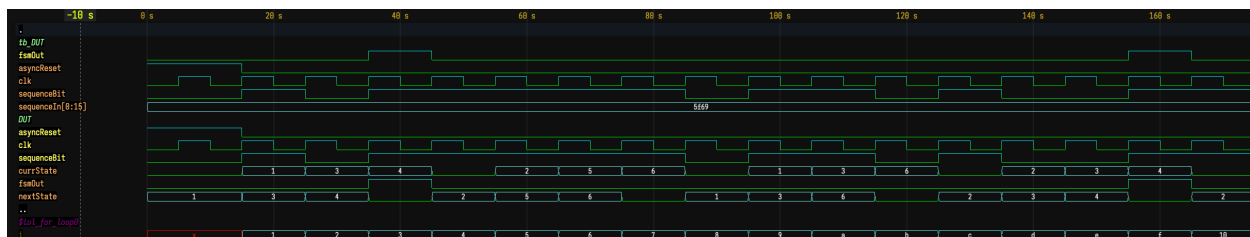


Figure 3.1. Waveform of the sequence 0101_1111_0110_1001

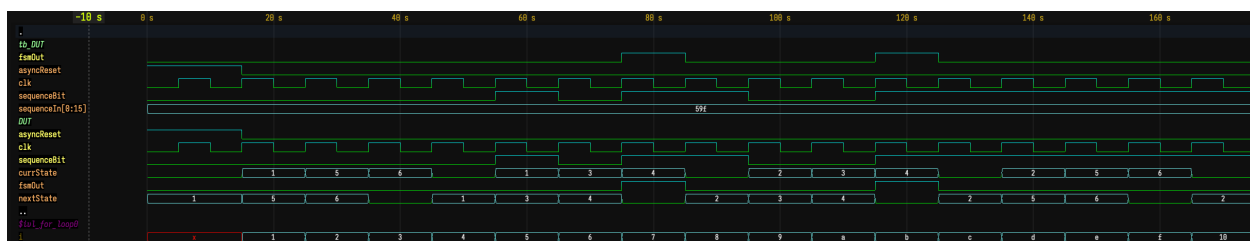


Figure 3.2. Waveform of the sequence 0000_0101_1001_1111



Figure 3.3. Waveform of the sequence 1111_0000_1000_0110

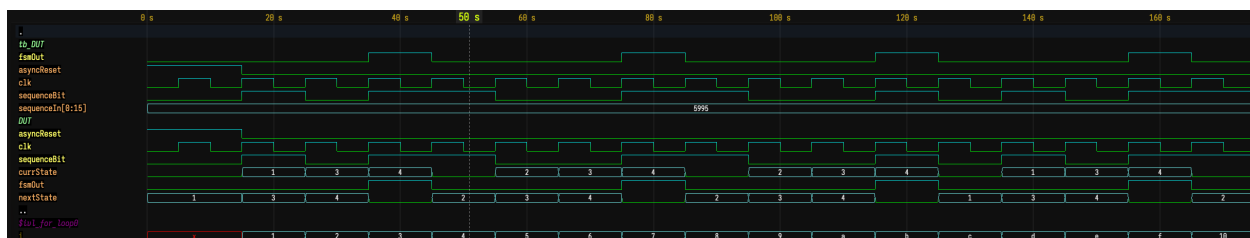


Figure 3.4. Waveform of the sequence 0101_1001_1001_0101

1. Problem Statement

In today's world people are required to carry numerous physical keys to regularly, or even daily, access locations such as their home, workplace, school or a post-box in a community area. Although physical keys are not necessarily cumbersome to carry, they are susceptible to, and often are, forgotten at home or even lost, leading to wasted time and requiring expensive services for a replacement. This is the problem I am seeking to solve through the use of a digital lock with the logic being implemented using a finite state machine (FSM) written in Verilog.

The user will be able to input a sequence of four bits to the FSM, which will then check the sequence against two predefined passwords. If the inputted sequence matches either of the passwords, the FSM will output a 1, allowing the user access to the location; if neither of the passwords were detected the output will be 0, denying the user access, after which the FSM will reset to allow the user to input a second sequence of bits.

Although the digital lock is more convenient than a physical key, there are some limitations of the design used; since the sequence is only 4 bits long and binary has only two states, the number of combinations for the password is only 16. Furthermore, the user is not limited to a certain number of attempts to input the correct password, therefore someone could enter all sixteen combinations and brute force the lock in a very short period. The first limitation could be alleviated by requiring the input sequence and password to be in decimal, therefore increasing the number of states of the password or by increasing the length of each password. Implementing either of these improvements would make the lock far more secure, therefore making the almost nullifying an attack by brute force, decreasing the probability that someone is likely to gain access unless they are permitted to, although incorporating these improvements would greatly increase the complexity of the FSM.

2. The FSM Design

The type of FSM I have chosen to design is a mealy machine. Its input, sequenceIn, is the string of bits the user will enter to gain access to the location or object the lock is ~~is~~ securing; the output, fsmOut, is 1 if the user entered a string of bits matching one of the correct passwords (the passwords being 0101 and 1001), and 0 for any other entered sequences. The FSM has seven states labelled a through g, the first state is called idle, this is the ~~reset~~ state reset state, which occurs when the FSM first starts or when a sequence of four bits is entered and then is reset for the next sequence of four bits. The next two states are found0 and found1, these are each triggered when a 0 or 1 is ~~enter~~ entered as the first bit of a sequence respectively. The fourth state is called found01_10, this state is triggered when either 1 or 0 is entered when the previous state is found0 and found1 respectively. The fifth state is found01_101 found010_100 is triggered when a 0 is inputted when the FSM is in the state found01_10 (the values chosen for the passwords are convenient when designing the FSM as it decreases the number of states required, due to each of the passwords find two bits being the same). If a 1 is entered when the FSM is in this state fsmOut will become 1 therefore letting the user enter the location the lock was protecting. The final two states are foundNull and foundNull2 either of these states will be triggered when the ~~an~~ inputted bits will not match either of the passwords, therefore these states act as ~~wait~~ wait states for the four bit sequence to finish.

For the first step of synthesising the FSM, I drew a table of the ~~states~~ states of the FSM, shown in table 1, which described the required sequence to be inputted to enter each state and assigned each state a letter a through g. Additionally I drew a state diagram, shown in Fig. 2.1, displaying each of the states of the FSM as circles with arrows between the states showing the flow of the FSM, this diagram also displays the input to enter one state from another.

beside the arrows, and the value of fsmOut when exiting that state, shown underneath the input value beside the arrows.

To check the design of the FSM and that the state transition diagram is correct, I subjected the FSM to a stream of inputs that contained both passwords, each appearing once, with the rest of the bits of the sequence being random to check that the FSM doesn't produce any false positives, additionally I traced the state transitions and the output of the FSM as each bit was entered. The result of this test can be seen in Fig. 2.2, the FSM performed as expected with the output changing to 1 after the first and third group of four bits was entered and remaining as 0 after the second and fourth group of bits.

Next I assigned each of the states to a binary code, as shown in table II, this is useful for the further synthesis of the FSM but it also reveals that three d-flip-flops are required for the circuit of the FSM due to the final state requiring 3 three bits in the binary code, each d-flip-flop will store one bit. Using the binary code and state diagram, I constructed a state transition table, shown shown in table III. The table displays what the next state and output will be depending on the current state and input. This table can be used in a similar way to a truth table to design the final circuit of the FSM, this is the reason for the final state 'h' which is there to fill the binary code, with each x being treated as a 'don't care' for use in Karnaugh maps.

Using the state transition table and Karnaugh maps, shown in Fig. 2.3. (created with the help of an online tool [4]). I minimised the logic for nextState and the output, taking A, B, C and x as inputs. By using the Karnaugh maps and a computer program [5] to help arrange the minimised circuit, I drew the circuit FSM circuit in Fig. 2.4

Finally, using the state transition table, shown in table 111, the logic of the FSM can be checked to verify the timing circuit is correct. By taking any output, e.g. A , as when a bit is 0 (or taking A as 1) in the state transition table, the table can be used to verify the outputs are correct.

3. DUT Verification

To verify the design under test (DUT), I used four distinct sequences for the input, sequenceIn; one of the sequences contained both passwords, one at the start of the sequence and one at the end (0101_1111_0110_1001). The second sequence contained both passwords one after the other after the first four bits (0000_0101_1001_1111). The final two sequences contained neither at the password (1111_0000_1000_0110) and each password appearing twice, respectively (0101_1001_1001_0101).

I believe these four sequences provide a sample of inputs that sufficiently demonstrate the DUT is functioning without verifying the output of all possible combinations of inputs. These four sequences contain groups of four bits that cause the FSM to enter every state in any way it can enter or exit them, additionally, I have decided to vary the position the passwords to ensure the FSM can handle the password being the first group in the sequence or the last etc. Therefore, I believe these inputs alone will demonstrate that the FSM is robust and works as intended.

Viewing the first waveform, shown in Fig. 3.1, generated by the different input 0101_1111_0110_1001, it can be seen that fsmOut is 1 as intended when the final bit of the first group is processed by the FSM, and that the output is 0 for the rest of the waveform except when the final final bit is processed changing the output back to 1, this shows that the FSM responds correctly when either at the two passwords are entered and when they are absent. Furthermore, by looking at the state or nextState at the positive edge of each clock cycle it is evident that nextState is correctly transitioning to each state when it is supposed to, this will be true for all other waveforms in this report.

Viewing the second ~~wave~~ waveform, shown in Fig. 3.2, generated by the sequence 0000-0101-1001-1111, it shows that fsmOut is 0 for the first four bits, then once the four bits that match the password are entered it changes to 1 then back to 0 then changes to 1 again, for the next four bits which match the second password, then returns to 0 for the rest of the simulation. The waveforms shown in Fig. 3.1 and Fig. 3.2 are useful when verifying the DUT as they reasonably show that the FSM can process and react correctly when the passwords are inputted, regardless of their position in the input sequence.

Viewing the third waveform, shown in Fig. 3.3, generated by the sequence 1111-0000-1000-0110, fsmOut is 0 for the entire simulation as intended, due to the sequence containing neither of the passwords. Viewing the ~~second~~ fourth waveform, shown in Fig. 3.4, generated by the sequence 0101-1001-1001-0101, fsmOut changes between 1 and 0 after each group of four bits have been entered due to each group of four bits being the password.

References

- [1] A. Khalid. ECS 1005. Class Lecture, Topic: “Finite State machines”
School of Electronics, Electrical Engineering and Computer Science, Queen’s
University Belfast, Northern Ireland, 26 Jan., 2025.
- [2] A. Khalid. ECS 1005. Class Laboratory, Topic: “Verilog Lab 6: Modelling a Finite
state machine” School of Electronics, Electrical Engineering and Computer
Science, Queen’s University Belfast, Northern Ireland, 27 Jan., 2026.
- [3] B. J. LaMeres, *Introduction to Logic Circuits & Logic Design with Verilog*.
Switzerland: Springer Cham, 2024.
- [4] “Logic circuit simplification (SOP and POS)”. [Online].
Available: <http://www.32x8.com>. [Accessed Mar. 6, 2026].
- [5] C. Burch, Logisim. [.tar.gz file]. 2014